

Containerlab Network Automation Sandbox — Setup Guide

FOR
DUMMIES

This repo gets you from “empty vCenter resource pool” to a working Debian VM running Docker + Containerlab + an Arista cEOS topology + a Python automation environment, so you can practice OSPF/BGP/IS-IS/IPSec/VXLAN and network automation against real (virtualized) EOS instances.

The automation story here leads with Nornir, NAPALM, and Netmiko — pure Python, no Ansible infrastructure required to get value on day one. Ansible tooling is also included and fully working, parked as an optional path for later if/when there’s appetite for the bigger investment (collections, Vault, playbook structure). See [Two automation paths](#) for why it’s laid out this way.

It’s split into two parts:

1. Create the VM — one-time, per person.
 - Corporate vCenter/ESXi: [docs/vcenter-vm-setup.md](#)
 - Proxmox home lab: [docs/proxmox-vm-setup.md](#) (use a VM, not an LXC — that doc explains why)
2. In-VM setup — run `./setup.sh` and answer a few questions. See [Quick Start](#).

Getting these files onto the VM (git / scp / zip): [docs/distributing.md](#).

The scripts are hypervisor-agnostic — they detect the network interface name rather than assuming VMware’s `ens192`, and handle both Debian’s classic `.list` and modern `deb822.sources` apt formats.

New here? Take the course instead

Don’t start with this README — start with [course/README.md](#). It’s a guided, module-by-module path built for Aston engineers: platform build → Containerlab basics → a spine-leaf fabric configured by hand → the same fabric automated with Nornir/NAPALM → a VXLAN+EVPN overlay with an automation capstone → secrets management. Its [Module 00](#) is a map of this entire repo — every file explained, plus a decision tree for “I want to change X, which file do I touch, which command do I run?” — worth reading even if you skip everything else.

This README is the *reference*. The course is the *path*.

What you end up with

- Debian 13 (Trixie) VM, routable on the work WiFi /23 subnet (no NAT, no jump hosts)

- Docker CE
- Containerlab (latest release, auto-detected by the official installer)
- A working directory layout under `~/labs`
- A Python virtualenv with Nornir, NAPALM, Netmiko, and Ansible — built with `uv` if available, falling back to plain `pip` otherwise (see [Tooling choices](#))
- A ready-to-deploy 2-node cEOS test topology to confirm everything works end-to-end before you build anything bigger
- A four-script progressive automation demo (`automation/nornir/`) that needs **zero credential setup** to run, because containerlab already bakes cEOS's default creds into its auto-generated inventory — plus three working secrets backends (`.env`, `pass`, `sops+age`) to demonstrate when you're ready to stop trusting defaults
- A parallel Ansible setup (`automation/ansible/`) with proper Vault-based secrets management, ready when you want it

What this does NOT automate

- **Downloading the vendor images.** Arista's cEOS portal (and Fortinet's) sits behind an authenticated account per their EULAs, so there's no way to script the download legally or reliably. Grab them from the [team Drive](#) or the vendor directly, then `scp` to the VM — see [Getting the images](#).
- **vCenter VM creation.** vSphere doesn't expose a good non-interactive path for this without `govc/PowerCLI/Terraform`, which is more setup than it's worth for a handful of lab VMs. Follow `docs/vcenter-vm-setup.md` instead — it's about 10 minutes of clicking.

Quick Start

Assumes you've already built the VM per `docs/vcenter-vm-setup.md` and can SSH into it.

On a minimal/netinst Debian install, check one thing first. The netinst image ships almost nothing, and if you set a root password during installation, Debian does *not* add your user to the `sudo` group. `setup.sh` checks for both and tells you how to fix it, but you can get ahead of it:

```
# If `sudo -v` fails or sudo isn't installed at all:
su -
apt-get update && apt-get install -y sudo
usermod -aG sudo <your-user>
exit
# then log out and back in - group changes need a new login
```

Everything else (`curl`, `gnupg`, `ca-certificates`, `iproute2`) is installed automatically by `setup.sh` before it needs them.

```
# 1. Get these files onto the VM (git clone, scp, or unzip -
# see docs/distributing.md for all three)
git clone <your-internal-git-url> ~/clab-bootstrap
cd ~/clab-bootstrap
```

```
# 2. Run the guided setup – it asks what you want and explains as it goes
chmod +x setup.sh lab scripts/*.sh
./setup.sh
```

That's it. `setup.sh` walks through everything below interactively: installing Docker/Containerlab/Python tooling, importing the cEOS image, deploying a test topology, and picking an automation path.

Re-running is the normal case. Every step checks real state before acting, so a second run is a no-op and a third is identical to the second. It's the update path after a `git pull`, and the repair path after a run that died halfway — it detects and rebuilds a half-built venv, restarts a stopped Docker daemon, and disables apt repos whose signing key went missing. Setup opens with a survey of what's already in place so you can see what it intends to do before it does it.

Watching it work

Long steps show a live status line: spinner, elapsed time, and the most recent line of output, redrawn in place. The clock keeps ticking even when a command is silent, so a slow step never looks like a hung one.

<code>./setup.sh</code>	default — one live status line per step
<code>CLAB_VERBOSE=1 ./setup.sh</code>	stream every command's full output; use when a step fails
<code>CLAB_QUIET=1 ./setup.sh</code>	step titles and timings only, no per-line detail

Full output always lands in `/tmp/clab-setup.log` regardless of mode, and the last 40 lines are printed automatically if a step fails.

Prefer to drive manually? Everything `setup.sh` does is just the numbered scripts, runnable directly:

```
./scripts/00-bootstrap.sh                # Docker, Containerlab, Python env
./scripts/01-import-ceos.sh ~/cEOS64-lab-<version>.tar.xz # verify + import cEOS
./scripts/02-deploy-lab.sh                # deploy the test topology
./automation/link-inventory.sh ~/labs/topologies/clab-testlab
```

Log out and back in after the first run (or `newgrp docker && newgrp clab_admins`) so your group memberships take effect.

Then run the automation demo:

```
netauto # activates the automation venv (alias added to ~/.bashrc)
cd automation/nornir
python3 01_netmiko_hello.py
python3 02_nornir_scale.py
python3 03_napalm_facts.py
python3 04_idempotent_deploy.py
python3 04_idempotent_deploy.py # run again – this time nothing changes
```

`automation/nornir/README.md` walks through what each script demonstrates and why — that's the one worth reading before showing this to coworkers. Ansible is available too, whenever you want it — see [Two automation paths](#).

Getting the images

Two images can't be fetched by any script: Arista cEOS and, if you're running the FortiGate topology, the FortiGate qcow2. Both sit behind vendor accounts.

Team Drive (fastest)

https://drive.google.com/drive/folders/1kBDv_xgv4T4NQJfWZtLKkCnYbmcfs9KU?usp=drive_link

```
ContainerLab/Arista Images/
  cEOS64-lab-4.34.7M.tar.xz          <- the image
  cEOS64-lab-4.34.7M.tar.xz.sha512sum <- grab this too, the import verifies it
  cEOS64-lab-4.34.7M.tar.xz.md5sum
  cEOS64-lab-4.34.7M.tar.xz.json
  cEOS64-lab-4.34.7M.tar.xz.cms.pem
```

```
ContainerLab/Fortigate Images/
  fortinet-FGT-v7.0.9-build0444.qcow2
```

Download the `.tar.xz` and its `.sha512sum` into the same directory — `01-import-ceos.sh` re-verifies the hash and warns if the checksum file is absent.

This mirror is for people who already hold the relevant Arista and Fortinet entitlements; it's a convenience, not a substitute for having one.

Or straight from the vendor

1. On your own laptop (not the VM), log into [Arista Software Downloads](#) with your Arista account.
2. Software Downloads → cEOS-lab → download the latest 64-bit `.tar.xz` and its matching `.sha512sum` file.
3. Verify the download is intact before you burn bandwidth pushing a bad file to the VM:

```
shasum -a 512 -c cEOS64-lab-<version>.tar.xz.sha512sum
# must print OK
```

FortiGate images come from <https://support.fortinet.com> → Download → VM Images → the KVM image.

Getting it onto the VM

```
scp cEOS64-lab-<version>.tar.xz cEOS64-lab-<version>.tar.xz.sha512sum yourname@<VM-IP>:~/
```

Then on the VM, `setup.sh` finds it automatically, or run the import directly:

```
./scripts/01-import-ceos.sh ~/cEOS64-lab-<version>.tar.xz
```

It re-verifies the hash (catching corruption from the transfer itself) and imports with a progress bar.

FortiGate is a different workflow — it's a VM image, not a container, so it has to be built with `vrnetlab` and needs nested virtualization enabled on the hypervisor (on vSphere that's **Expose hardware assisted virtualization to the guest OS**, VM powered off). One command does the whole thing:

```
./lab fortigate
```

It checks `/dev/kvm` first and names the exact hypervisor setting if it's missing, then clones `vrnetlab`, builds, and reconciles your topology's `image:` tag with whatever the build produced. Full walkthrough and manual steps: [docs/vrnetlab-fortigate.md](https://docs.vrnetlab-fortigate.md).

Image tags

Because everyone downloads their own build, the cEOS version differs per person — so the topologies reference `ceos:latest` rather than a pinned version. `01-import-ceos.sh` tags each import as both `ceos:<version>` and `ceos:latest`, so `docker images` still shows exactly what you have while the topologies keep working across upgrades.

`02-deploy-lab.sh` checks the topology's image exists locally before deploying. This matters because `containerlab` treats a missing image as “pull it from a registry”, and cEOS is on no public registry — so a tag mismatch surfaces as:

```
pull access denied for ceos, repository does not exist or may require 'docker login'
```

which looks like an authentication problem but is always local. The preflight reports the real cause and lists what you actually have imported. If `ceos:latest` is missing but exactly one cEOS image exists, it adopts that one automatically.

To pin a specific version instead, set `image: ceos:<version>` in the topology — the preflight validates that too.

Driving the lab

`./lab` is the day-to-day tool. `setup.sh` builds the platform once; `./lab` is what you run every time after that. Run it bare for a menu, or give it a verb:

```
./lab          # menu – pick from a list
./lab status   # what's running right now
./lab deploy   # bring the lab up
./lab ssh      # log into a node (user admin, password admin)
./lab cli      # EOS CLI without SSH – works when SSH doesn't
./lab graph    # topology diagram in a browser
./lab destroy  # tear it down (cEOS nodes eat RAM)
./lab topo     # switch which topology the above act on
./lab learn    # where to go next in the course
```

It remembers which topology you selected, so `./lab graph` after `./lab topo spine-leaf.clab.yml` does the right thing.

`setup.sh` stops at a natural checkpoint once the test topology deploys and asks whether you want to continue into the automation walkthrough. Answering no is a normal, complete outcome — it prints the same “what now?” card and you carry on with `./lab`. Re-run `./setup.sh` whenever you want the rest; it skips everything already done.

The underlying script

`./lab` wraps `02-deploy-lab.sh`, which you can also drive directly:

```
./scripts/02-deploy-lab.sh      # deploy the test topology (default)
./scripts/02-deploy-lab.sh inspect # what's running, and node IPs
```

```
./scripts/02-deploy-lab.sh graph      # topology diagram in a browser
./scripts/02-deploy-lab.sh destroy   # tear it down (cEOS nodes eat RAM)
```

The action can be the only argument, or you can name a topology first — `./scripts/02-deploy-lab.sh spine-leaf.clab.yml graph`. A bare filename is resolved against `~/labs/topologies/` and the repo's `topologies/`.

`graph` starts containerlab's built-in web server on port 50080 and runs in the foreground until you Ctrl-C. It prints the VM's actual address, so over SSH just open the URL it gives you. Graphing reads the `.clab.yml` and doesn't require a running lab — deploying first only makes node state meaningful. If the page won't load from your laptop, check that nothing is filtering 50080 between you and the VM.

To confirm the lab is up without the browser:

```
sudo containerlab inspect -t ~/labs/topologies/testlab.clab.yml
docker ps --filter name=clab-testlab
```

Two automation paths

Nornir / NAPALM / Netmiko (`automation/nornir/`) — **start here**. Pure Python, no framework-specific YAML DSL to learn, no collections to install, and — specific to this lab — no credentials to set up at all, because containerlab already bakes cEOS's default `admin/admin` into its auto-generated inventory. The four scripts in there walk the same Netmiko → Nornir → NAPALM → idempotent-deploy progression as the team's automation course, just retargeted from GNS3 + a bastion-host VM onto this Containerlab topology. This is the version worth demoing to coworkers who haven't seen network automation before: the distance from “nothing installed” to “watch this self-heal a config drift” is about five minutes.

Ansible (`automation/ansible/`) — **available, not required**. Fully working, including proper Ansible Vault secrets management (see [Secrets management](#)). Ansible's real strength — a huge module ecosystem, idempotent primitives built into every module, agentless execution — pays for itself once an org has existing playbooks, roles, and people fluent in the YAML structure, or is running something like AWX/Ansible Automation Platform. Standing all of that up just to prove automation works at all is a lot of ceremony for a first demo, which is why it's not the default path here. It's kept fully set up in this repo specifically so it's a config away, not a rewrite, whenever that investment makes sense for the team.

Both paths point at the same containerlab-generated inventory (`automation/link-inventory.sh` refreshes both at once), so nothing about the lab itself changes depending on which tool you reach for.

Tooling choices

Worth calling out explicitly, since a couple of these are as much a “here's a good Linux tool” moment for coworkers as they are project decisions.

uv instead of plain pip + `venv`. **uv** is a single Rust-native binary (from the makers of Ruff) that replaces `pip + venv + pip-tools + pyenv` with one fast tool. `00-bootstrap.sh` installs it automatically (`curl -LsSf https://astral.sh/uv/install.sh | sh`) and uses `uv pip install -r requirements.txt` to build the `venv` at `~/venvs/netauto` — dramatically faster than `pip` on a resource-constrained lab VM, especially reinstalling `ansible-core` and its dependency tree.

If `uv` can't be installed (offline box, locked-down network policy), the script falls back automatically to `python3 -m venv + plain pip` — same `venv` layout either way, so `netauto` activates it the same way regardless of which tool built it.

There's also a full `uv` project mode available if you'd rather work that way day to day, via `automation/pyproject.toml`:

```
cd automation
uv sync          # creates automation/.venv/, resolves deps, writes uv.lock
uv run python3 nornir/02_nornir_scale.py  # runs in that venv, no activation needed
```

`automation/requirements.txt` still exists alongside `pyproject.toml` purely as the `pip`-only fallback path `00-bootstrap.sh` uses when `uv` isn't available — hand-kept in sync with `pyproject.toml`'s dependency list.

`gum` for the setup UI. `setup.sh` offers to install `gum` — a small Go binary from Charm that gives shell scripts real menus, spinners, and styled prompts. It's installed from Charm's own apt repo (Debian's repos don't carry it).

Entirely optional and purely cosmetic: `lib/ui.sh` implements every UI function twice, once using `gum` and once in plain bash (`read` prompts, a braille spinner, ANSI colors). The fallback isn't an afterthought — it has to work, because `setup.sh` runs before anything is installed and can't assume `gum` exists yet.

Secrets backends. See [Secrets management](#) below — three different tools, deliberately, so there's a real comparison to show coworkers rather than one prescribed answer.

Repo layout

```
clab-bootstrap/
├─ setup.sh          <- START HERE (once) – guided platform setup
├─ lab               <- START HERE (daily) – deploy/ssh/graph/destroy
├─ README.md         <- you are here
├─ course/           <- 📖 the guided course (modules 00–06)
│   └─ README.md      <- course overview + module index
│   └─ 00-orientation.md <- FILE MAP + DECISION TREE – read this when lost
│   └─ 01-platform.md  <- VM + setup.sh
│   └─ 02-containerlab-basics.md <- the lab lifecycle, on the 2-node testlab
│   └─ 03-spine-leaf.md  <- Clos fabric + OSPF underlay, by hand
│   └─ 04-automation.md  <- same underlay via Nornir/NAPALM + bespoke script
│   └─ 05-vxlan-evpn.md  <- overlay + automation capstone
│   └─ 06-secrets.md     <- credential management tour
├─ lib/ui.sh         <- terminal UI helpers (gum, w/ plain-bash fallback)
├─ docs/
│   └─ vcenter-vm-setup.md <- vCenter/vSphere steps (incl. nested virt toggle)
│   └─ proxmox-vm-setup.md <- Proxmox home-lab steps (+ why VM not LXC)
│   └─ vrnetlab-fortigate.md <- building VM-based nodes (FortiGate) with vrnetlab
│   └─ distributing.md     <- git vs scp vs zip, for sharing with the team
├─ scripts/
│   └─ 00-bootstrap.sh    <- OS + Docker + Containerlab + venv (uv, w/ pip fallback)
│   └─ 01-import-ceos.sh  <- verify + docker import cEOS (tags ceos:latest)
│   └─ 02-deploy-lab.sh   <- deploy / destroy / inspect / graph a topology
│   └─ 03-build-vrnetlab.sh <- build FortiGate (or other VM images) via vrnetlab
│   └─ sync-labs.sh       <- keep ~/labs/topologies in step with the repo
├─ topologies/
│   └─ testlab.clab.yml    <- 2-node back-to-back cEOS smoke test
│   └─ spine-leaf.clab.yml <- 2-spine/2-leaf Clos fabric (course modules 03–05)
│   └─ memory-test.clab.yml <- 10-node mixed lab: 2x FortiGate + 4x cEOS + FRR/hosts
├─ automation/
│   └─ requirements.txt    <- pip fallback (pyproject.toml is uv's source of truth)
│   └─ pyproject.toml     <- uv project manifest – `cd automation && uv sync`
```

```

├─ link-inventory.sh      <- symlinks containerlab's generated inventory in
├─ lib/vault.py          <- shared Ansible Vault decryptor
├─ nornir/               <- START HERE – zero-setup demo path
│   ├── README.md        <- walkthrough + "what to point out to coworkers"
│   ├── SECRETS.md       <- .env vs pass vs sops+age, compared in depth
│   ├── config.yaml
│   ├── creds.py         <- picks up NORNIR_USERNAME/PASSWORD if set, else no-op
│   ├── .env.example
│   ├── secrets.example.yaml <- template for the sops+age-encrypted secrets file
│   ├── run-with-env.sh   <- wrapper: inject creds from .env
│   ├── run-with-pass.sh  <- wrapper: inject creds from `pass`
│   ├── run-with-sops.sh  <- wrapper: inject creds from sops+age
│   ├── 01_netmiko_hello.py
│   ├── 02_nornir_scale.py
│   ├── 03_napalm_facts.py
│   ├── 04_idempotent_deploy.py
│   ├── 05_spineleaf_underlay.py <- deploys the fabric underlay (course module 04)
│   ├── templates/ceos_base.j2
│   ├── templates/spineleaf_underlay.j2
│   └─ nornir-simple-inventory.yml <- symlink, created by link-inventory.sh (not in git)
├─ ansible/              <- optional, available when you want it
│   ├── ansible.cfg
│   ├── requirements.yml  <- Galaxy collections: arista.eos, ansible.netcommon
│   ├── group_vars/ceos/
│   │   ├── vars.yml      <- plaintext connection settings
│   │   └─ vault.yml.example <- copy to vault.yml, fill in, then `ansible-vault encrypt`
│   ├── inventory.yml     <- symlink, created by link-inventory.sh (not in git)
│   └─ playbooks/gather_facts.yml

```

Inventory files are never committed or maintained by hand — containerlab regenerates them fresh in the `clab-<labname>/` directory every time you deploy, and `automation/link-inventory.sh` points both Nornir and Ansible at whichever lab you last deployed.

Secrets management

Four approaches live in this repo, deliberately spanning “zero ceremony” to “production-grade” — worth showing coworkers side by side as a “how much tooling do you actually need” comparison rather than a single prescribed answer. Full detail (install steps, day-to-day commands, when each earns its keep) is in [automation/nornir/SECRETS.md](#); short version:

Backend	Where	Ceremony	Good for
.env file	automation/nornir/run-with-env.sh	None — plaintext, gitignored	Solo lab, fastest demo
pass	automation/nornir/run-with-pass.sh	GPG key + keyring	Individuals/teams already using GPG
sops + age	automation/nornir/run-with-sops.sh	age keypair, no keyring ceremony	Diff-friendly, git-committable, closest to real GitOps/CI pipelines
Ansible Vault	automation/ansible/	Vault password, Galaxy collections	The Ansible path specifically; production-standard for Ansible shops

All three Nornir-path backends work the same way under the hood: they set `NORNIR_USERNAME` / `NORNIR_PASSWORD` in the environment before a script runs, and `automation/nornir/creds.py` (imported by every numbered script) overrides the inventory defaults if it finds them set — otherwise it's a no-op and you get the zero-setup behavior described earlier.

Ansible Vault, briefly (full walkthrough was already worked out earlier in this repo's history and lives in `automation/ansible/`): an AES256-encrypted YAML file, decrypted with a password you're prompted for at runtime (`--ask-vault-pass`). One-time setup from `automation/ansible/group_vars/ceos/`:

```
cp vault.yml.example vault.yml
nano vault.yml          # fill in real credentials
ansible-vault encrypt vault.yml
```

then `ansible-playbook --ask-vault-pass playbooks/gather_facts.yml`. Day to day: `ansible-vault view|edit|rekey vault.yml`. For unattended/CI runs, `ANSIBLE_VAULT_PASSWORD_FILE` (gitignored) or `--vault-password-file` instead of typing it. `automation/lib/vault.py` even lets a Nornir script decrypt that same vault file, if you want to mix approaches.

Where all of these diverge from a large enterprise: none of the four gives you credential rotation, per-user audit trails, or short-lived dynamic secrets on their own. Real shops back this with a centralized secrets manager — HashiCorp Vault, CyberArk, Thycotic/Delinea, or a cloud provider's secrets manager — with CI pipelines pulling credentials at run time (`sops` in particular is usually paired with a cloud KMS instead of `age` at that scale, same workflow, different backend). The underlying discipline — reference a variable name, never a literal secret, in anything that gets committed — is the same either way; swapping the backend later is a config change, not a rewrite.

Distributing this to the team

Three options — internal git repo (recommended), `scp`, or a zip file — with full step-by-step for each in [docs/distributing.md](#).

Short version, if you have an internal git host:

```
git init && git add . && git commit -m "Containerlab automation lab bootstrap"
git remote add origin <your-internal-git-url>
git push -u origin main
```

Then each coworker runs, on their own VM:

```
git clone <your-internal-git-url> ~/clab-bootstrap
cd ~/clab-bootstrap && ./setup.sh
```

Updates later are `git pull && ./setup.sh` — the setup script skips whatever's already done.

Notes on the original build

The reference build used Debian 13 (Trixie). At the time, Docker's repo only published `bookworm` packages for Trixie hosts and Containerlab had to be pinned, which caused some install friction. As of this writing Docker publishes packages directly under the `trixie` codename and Containerlab's installer (`get.containerlab.dev`) auto-detects the latest release, so `00-bootstrap.sh` uses the native `trixie` repo by default. If Docker repo 404s on your build date, edit the `DOCKER_CODENAME` variable near the top of `00-bootstrap.sh` and set it to `bookworm` as a fallback — that's the workaround that was needed previously.

The automation approach itself also traces back further, to a team automation course originally run against a GNS3 topology reached through a bastion-host VM (Ubuntu jumpbox + DNAT on an edge router). The Netmiko → Nornir → NAPALM progression in `automation/nornir/` mirrors that course directly; only the underlying lab platform changed, from GNS3 + bastion to Containerlab running locally on this VM.

Troubleshooting

Symptom	Cause / Fix
<code>ifdown: dhcpcd is not running</code>	Benign — that's the DHCP <i>server</i> daemon, not the client. Ignore it.
No DHCP lease	Confirm the VM's NIC is attached to a bridge/port group with DHCP — <code>vmbrio</code> on Proxmox, the routable port group on VMware — and that you picked VirtIO (Proxmox) or VMXNET3 (VMware). Try <code>sudo dhclient -v <iface></code> .
<code>apt</code> warns “configured multiple times”	Duplicate repo entries. Check for a stray <code>/etc/apt/sources.list.d/trixie.list</code> alongside the stock <code>debian.sources</code> and delete the one you didn't intend.
<code>docker: 404</code> on <code>apt</code> update	Docker hasn't published your release yet. <code>00-bootstrap.sh</code> detects the codename from <code>/etc/os-release</code> and retries against bookworm automatically; force it with <code>DOCKER_CODENAME=bookworm ./scripts/00-bootstrap.sh</code> .
<code>sudo: command not found</code> , or “user is not in the sudoers file”	Debian's installer skips <code>sudo</code> and leaves your user out of the <code>sudo</code> group when you set a root password. See the Quick Start note above.
Spinner spins forever, never finishes	Something under the spinner is waiting on input you can't see. Setup primes <code>sudo</code> up front to prevent this; if it recurs, re-run with <code>CLAB_VERBOSE=1 ./setup.sh</code> to see the prompt.
A step fails and you can't tell why	<code>CLAB_VERBOSE=1 ./setup.sh</code> skips the spinner and streams every command's output live. Or run the underlying script directly: <code>bash scripts/00-bootstrap.sh</code> .
<code>E: The repository ... is not signed/Failed to parse keyring</code>	A repo file whose <code>signed-by=</code> key is missing — an interrupted install can leave <code>charm.list</code> or <code>docker.list</code> behind without its keyring. This breaks every <code>apt-get update</code> on the box, not just this script's. <code>setup.sh</code> now detects it during preflight and offers to disable the file; to fix by hand, <code>sudo rm /etc/apt/sources.list.d/<name>.list</code> .
<code>apt-get update</code> fails during bootstrap	Run <code>sudo apt-get update</code> by hand — the failing repo is named in the error. Then <code>ls /etc/apt/sources.list.d/</code> to see what's configured.
<code>ssh spine1</code> fails, “could not resolve hostname”	New shell session hasn't picked up <code>/etc/ssh/ssh_config.d/</code> . Log out/in, or use <code>sudo containerlab inspect -t <file></code> to

Symptom	Cause / Fix
	get the IP directly.
pull access denied for ceos, repository does not exist	Not an auth problem — the topology's <code>image: tag</code> doesn't match anything imported locally, so containerlab tried a registry. Run <code>docker images \ grep ceos</code> and either <code>docker tag <image> ceos:latest</code> or set the topology's <code>image: to match</code> . <code>02-deploy-lab.sh</code> now checks this up front.
Deploy uses an old topology after a <code>git pull</code>	<code>~/labs/topologies/</code> is the working copy <code>02-deploy-lab.sh</code> defaults to. The bootstrap syncs it from the repo, but leaves files you've edited alone (it says so when it does). Delete your copy to take the repo version.
cEOS import "hash mismatch"	Re-verify on your laptop first (<code>shasum -a 512 -c ...</code>); if that's OK, the corruption happened in transit — <code>re-scp</code> and don't let the laptop sleep mid-transfer.
<code>04_idempotent_deploy.py</code> shows a diff every run, never settles	cEOS eAPI config compare can be picky about exact syntax — check <code>templates/ceos_base.j2</code> renders valid EOS config by hand first (ssh in and paste it).
<code>uv: command not found</code> right after bootstrap	<code>uv</code> installs to <code>~/local/bin</code> ; open a new shell (or source <code>~/local/bashrc</code>) so <code>PATH</code> picks it up.
<code>sops: command not found</code>	No apt package on Debian — see automation/nornir/SECRETS.md for the <code>.deb-from-GitHub-releases</code> install.
<code>setup.sh: Permission denied</code>	<code>scp/unzip</code> didn't preserve the executable bit: <code>chmod +x setup.sh lab scripts/*.sh</code>
Setup failed partway through	Re-run <code>./setup.sh</code> — completed steps are detected and skipped. Full output is in <code>/tmp/clab-setup.log</code> .